

Parcial 1: Localización Falla Parque Solar

Luz Adriana Salazar Varón¹

¹ Pontificia Universidad Javeriana

¹luza_salazar@javeriana.edu.co

Resumen. El siguiente documento presenta el desarrollo del primer parcial de la materia Análisis de Algoritmos en el cual se plantea la solución al problema de identificación de paneles con déficit de producción energética para un parque de generación de energía solar, haciendo uso del método dividir y vencer, para lo cual se plantea un análisis del problema, diseño del algoritmo, descripción de la estrategia dividir y vencer, pseudocódigo y finalmente el análisis de complejidad.

1 Análisis del problema

La problemática consiste en que un parque de generación de energía solar representa o almacena información de sus paneles a través de dos matrices cuadradas E y P. La matriz E almacena el valor de la producción de energía esperada para cada panel correspondiente a la posición (i,j) , mientras que la matriz P almacena el valor de la producción real de energía para cada panel ubicado en la misma posición (i,j) .

Por otro lado, se establece una tolerancia τ que indica cuál es la máxima diferencia permitida entre el valor esperado y el valor real de la producción de energía de un panel. Es decir que, se considera que un panel tiene falla y debe ser reportado cuando la diferencia entre el valor esperado y el valor real de producción supera el valor de tolerancia τ establecido.

Las entradas del problema son las dos matrices E y P, que deben ser estrictamente cuadradas, del mismo tamaño y cuyos valores deben pertenecer al conjunto de los números reales, y la tolerancia τ , que debe pertenecer al conjunto de los números naturales, es decir, debe ser mayor o igual a 0. Mientras que, el resultado que debe retornar el algoritmo es una lista con las coordenadas (i,j) de todos los paneles cuya diferencia entre la producción de energía esperada y la real haya superado el valor de tolerancia τ .

Finalmente, los principales casos especiales que se pueden presentar son según la cantidad de paneles reportados con falla. El primero es que no haya ningún panel cuya pérdida de producción supere la tolerancia τ , por lo que el algoritmo deberá retornar una lista vacía. También puede presentarse el caso en el que solo haya un panel fuera de tolerancia, donde se deberá retornar únicamente sus coordenadas (i,j) . Por último, puede haber múltiples paneles que superen la tolerancia, por lo que el algoritmo deberá identificar y retornar las coordenadas de todos estos.

2 Diseño algorítmico

A continuación, se presenta el diseño del algoritmo propuesto para la solución junto a un ejemplo de su funcionamiento.

- **Entradas**
 - a. Dos matrices $(E, P) \in \mathbb{R}^{n \times n}$, donde n representa el tamaño de las matrices cuadradas y todos sus valores pertenecen al conjunto de los reales. Ambas matrices deberán tener exactamente el mismo tamaño y deberán contener al menos un valor.
 - b. Tolerancia $\tau \in \mathbb{N}$, es decir que es un valor numérico que debe ser mayor o igual a 0.
- **Salida**
 - a. Lista de pares que contenga todas las coordenadas de los paneles reportados con falla. Por lo tanto, todos los valores (i, j) deberán pertenecer a los naturales y ninguno deberá superar el rango de las matrices recibidas. Reportados = $[(i_1, j_1), (i_2, j_2), (i_3, j_3), \dots]$, donde $i \leq n-1$ y $j \leq n-1$.

Teniendo esto en cuenta, el funcionamiento general del algoritmo consistirá en utilizar la recursión para dividir la matriz inicial en cuatro submatrices cuadradas. Cada una de estas repetirá el mismo proceso de división hasta llegar a una matriz de una sola posición, donde se realizará el cálculo de la diferencia entre la producción de energía esperada y la real de ese panel.

En caso de que esta diferencia supere la tolerancia τ , se retornarán las coordenadas de dicho panel. Los demás llamados recursivos se encargarán de crear una lista uniendo las coordenadas retornadas por sus nodos hijos directos, hasta obtener finalmente la lista con las coordenadas de todos los paneles fuera de tolerancia.

2.1 Ejemplo visual

En esta sección se muestra un ejemplo que permitirá observar el paso a paso del funcionamiento del algoritmo con una matriz de 8×8 .

Las siguientes son las matrices E y P propuestas para el ejercicio con una tolerancia $\tau = 3$, las casillas sombreadas corresponden a los paneles que superan la tolerancia:

E =	5	8	4	7	16	6	3	9
	13	6	9	5	8	10	6	4
	4	9	13	7	5	8	10	7
	8	5	6	10	7	4	9	5
	9	7	5	9	6	10	4	7
	6	11	8	5	12	7	6	4
	3	7	12	6	2	8	9	6
	8	4	7	9	6	5	10	7

P =	4	3	3	5	10	5	2	9
	5	4	2	1	8	7	1	3
	3	7	6	7	4	5	9	6
	7	4	5	10	6	3	5	4
	8	6	4	6	2	15	3	5
	5	9	7	4	8	5	4	2
	2	6	10	5	4	6	8	2
	7	3	5	8	5	4	2	6

Ilustración 1 – Matrices E y P iniciales

La recursión funcionará dividiendo las matrices en cuatro submatrices utilizando sus índices. Inicialmente, se toma desde la fila y columna 0 hasta la fila y columna $n - 1$. Por lo tanto, al iniciar el algoritmo recibiría las dos matrices E y P, $\tau = 3$, $filaInicio = 0$, $filaFin = 7$, $columnaInicio = 0$ y $columnaFin = 7$.

Se comprueba si los índices de inicio y fin de las filas y columnas son iguales, es decir, si $filaInicio == filaFin$ y $columnaInicio == columnaFin$, para determinar si ya se llegó al caso recursivo de una sola posición. En este caso no se cumple, ya que $0 \neq 7$, por lo tanto, se dividen las matrices calculando los nuevos rangos a partir de la mitad de los índices recibidos.

Para este caso, la mitad de las filas y columnas es 3, por lo que se obtienen cuatro nuevas submatrices: la matriz superior izquierda, con $filaInicio = 0$, $filaFin = 3$, $columnaInicio = 0$ y $columnaFin = 3$; la matriz superior derecha, con $filaInicio = 0$, $filaFin = 3$, $columnaInicio = 4$ y $columnaFin = 7$; la matriz inferior izquierda, con $filaInicio = 4$, $filaFin = 7$, $columnaInicio = 0$ y $columnaFin = 3$; y la matriz inferior derecha, con $filaInicio = 4$, $filaFin = 7$, $columnaInicio = 4$ y $columnaFin = 7$.

E =	5	8	4	7	16	6	3	9
	13	6	9	5	8	10	6	4
	4	9	13	7	5	8	10	7
	8	5	6	10	7	4	9	5
	9	7	5	9	6	10	4	7
	6	11	8	5	12	7	6	4
	3	7	12	6	2	8	9	6
	8	4	7	9	6	5	10	7

P =	4	3	3	5	10	5	2	9
	5	4	2	1	8	7	1	3
	3	7	6	7	4	5	9	6
	7	4	5	10	6	3	5	4
	8	6	4	6	2	15	3	5
	5	9	7	4	8	5	4	2
	2	6	10	5	4	6	8	2
	7	3	5	8	5	4	2	6

Ilustración 2 - Primera división

El algoritmo comprueba nuevamente si los rangos recibidos corresponden a una sola posición, en este caso no se cumple y por lo tanto, cada llamado recursivo divide otra vez la matriz recibida en cuatro submatrices.

E =	5	8	4	7	16	6	3	9
	13	6	9	5	8	10	6	4
	4	9	13	7	5	8	10	7
	8	5	6	10	7	4	9	5
	9	7	5	9	6	10	4	7
	6	11	8	5	12	7	6	4
	3	7	12	6	2	8	9	6
	8	4	7	9	6	5	10	7

P =	4	3	3	5	10	5	2	9
	5	4	2	1	8	7	1	3
	3	7	6	7	4	5	9	6
	7	4	5	10	6	3	5	4
	8	6	4	6	2	15	3	5
	5	9	7	4	8	5	4	2
	2	6	10	5	4	6	8	2
	7	3	5	8	5	4	2	6

Ilustración 3 - Segunda división

Cada llamado recursivo comprueba nuevamente si los rangos recibidos corresponden a una sola posición, en este caso no se cumple y por lo tanto, cada llamado divide nuevamente la submatriz recibida en cuatro submatrices.

E =	5	8	4	7	16	6	3	9
	13	6	9	5	8	10	6	4
	4	9	13	7	5	8	10	7
	8	5	6	10	7	4	9	5
	9	7	5	9	6	10	4	7
	6	11	8	5	12	7	6	4
	3	7	12	6	2	8	9	6
	8	4	7	9	6	5	10	7

P =	4	3	3	5	10	5	2	9
	5	4	2	1	8	7	1	3
	3	7	6	7	4	5	9	6
	7	4	5	10	6	3	5	4
	8	6	4	6	2	15	3	5
	5	9	7	4	8	5	4	2
	2	6	10	5	4	6	8	2
	7	3	5	8	5	4	2	6

Ilustración 4 - Tercera división

Finalmente, se cumple la condición del caso base, ya que cada llamado recursivo recibe el mismo rango de una sola posición para la fila y la columna (*filaInicio* == *filaFin* && *columnaInicio* == *columnaFin*). Por lo tanto, se verifica si el panel ubicado en esa posición (i, j) se encuentra fuera de la tolerancia y, en caso de que esto se cumpla, se retornan las coordenadas de dicho panel. Si el panel se encuentra dentro de la tolerancia, se retorna una lista vacía.

A partir de esto, los llamados anteriores a los nodos hoja recibirán las listas retornadas por sus cuatro nodos hijos y se encargarán de unirlos en una sola lista. Este proceso se repite en cada nivel de la recursión, uniendo los resultados de las

submatrices hasta llegar al llamado inicial, que finalmente retornará una lista general con las coordenadas de todos los paneles fuera de tolerancia.

El siguiente diagrama ilustra cómo se crearían las listas de paneles reportados en cada nivel, desde los nodos hoja que retornan su coordenada si supera la tolerancia o vacío si no, hasta el nodo raíz que reúne la lista general.

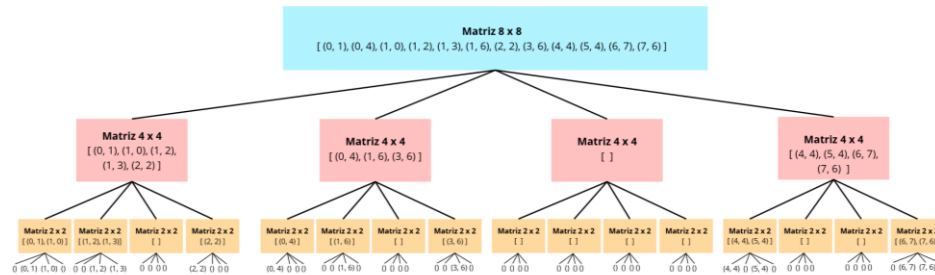


Ilustración 5 - Diagrama de recurrencia

3 Descripción de la estrategia Divide y Vencer

División: La matriz que se recibe se divide en cuatro submatrices cuadradas de menor tamaño. Este proceso se repite recursivamente para cada una de las submatrices hasta llegar a una matriz de una sola posición.

Conquista: El subproblema consiste en verificar si la región recibida corresponde a una sola posición, con el fin de determinar si se debe continuar con la división recursiva o si se ha llegado al caso base. La recursión se aplica a las cuatro submatrices generadas, por lo que no se descarta ninguna región durante el proceso.

Caso base: Se llega al caso base cuando el rango recibido corresponde a una matriz de una sola posición. En este caso, se verifica si la producción del panel ubicado en (i,j) está fuera de la tolerancia, es decir, si $E[i][j] - P[i][j] > \tau$. Si se cumple esta condición, se retorna la coordenada del panel; de lo contrario, se retorna una lista vacía.

Combinación: Cada nodo recibe las listas retornadas por sus cuatro hijos directos y las combina en una sola lista, que a su vez es retornada al nodo padre. Este proceso se repite en cada nivel de la recursión hasta llegar al llamado inicial, donde se obtiene la lista general con las coordenadas de todos los paneles reportados como defectuosos.

El algoritmo logra reportar todos los paneles fuera de tolerancia ya que no se descarta ninguna región y la recursión se detiene hasta llegar a cada panel por separado y verifica si cumple con la tolerancia. Debido a que aquellos paneles que no superan la tolerancia establecida retornan una lista vacía, se puede afirmar que nunca

se retorna un panel que esté dentro de la tolerancia. Finalmente, no se pierden paneles defectuosos pues no hay descarte de regiones.

4 Pseudocódigo

A continuación, se presenta el pseudocódigo que consta de dos funciones. La primera (*GenerarListaTolerancia*), se encarga de recibir los parámetros iniciales y calcular el rango inicial de posiciones con el que se realiza el primer llamado a la función recursiva (*BuscarDefectuosos*).

Esta segunda función es el algoritmo principal que aplica el procedimiento de dividir y vencer con la división de cuatro submatrices y la combinación de cada uno de sus resultados en una sola lista.

Algorithm 1 Búsqueda de paneles fuera de tolerancia

```

1: function GENERARLISTATOLERANCIA( $E, P, tolerancia$ )
2:    $n \leftarrow$  tamaño de  $E$ 
3:   return BUSCARDEFECTUOSOS( $E, P, tolerancia, 0, n - 1, 0, n - 1$ )
4: end function
5: function BUSCARDEFECTUOSOS( $E, P, tolerancia, filaInicio, filaFin, columnaInicio, columnaFin$ )
6:   if  $filaInicio = filaFin$  and  $columnaInicio = columnaFin$  then
7:     if  $E[filaInicio][columnaInicio] - P[filaInicio][columnaInicio] >$ 
        $tolerancia$  then
8:       return  $\{(filaInicio, columnaInicio)\}$ 
9:     else
10:      return  $\{\}$ 
11:    end if
12:   end if
13:    $mitadFila \leftarrow \lfloor (filaInicio + filaFin)/2 \rfloor$ 
14:    $mitadColumna \leftarrow \lfloor (columnaInicio + columnaFin)/2 \rfloor$ 
15:    $matrizSuperiorIzquierda \leftarrow$ 
     BUSCARDEFECTUOSOS( $E, P, tolerancia, filaInicio, mitadFila, columnaInicio, mitadColumna$ )
16:    $matrizSuperiorDerecha \leftarrow$ 
     BUSCARDEFECTUOSOS( $E, P, tolerancia, filaInicio, mitadFila, mitadColumna +$ 
1,  $columnaFin$ )
17:    $matrizInferiorIzquierda \leftarrow$ 
     BUSCARDEFECTUOSOS( $E, P, tolerancia, mitadFila$ 
1,  $filaFin, columnaInicio, mitadColumna$ )
18:    $matrizInferiorDerecha \leftarrow$ 
     BUSCARDEFECTUOSOS( $E, P, tolerancia, mitadFila$ 
1,  $filaFin, mitadColumna + 1, columnaFin$ )
19:   return  $matrizSuperiorIzquierda \cup matrizSuperiorDerecha \cup$ 
      $matrizInferiorIzquierda \cup matrizInferiorDerecha$ 
20: end function

```

Ilustración 6 - Pseudocódigo del algoritmo

Referencias

1. Baldonado, M.; Chang, C.-C.K.; Gravano, L.; Paepcke, A.: The Stanford Digital Library Metadata Architecture. *Proceedings of the 3rd International Conference on Web Information System and Technologies* (WEBIST 2007), pp.108–121 (1998)
2. Bruce, K.B.; Cardelli, L.; Pierce, B.C.: Comparing Object Encodings. Abadi, M.; Ito, T. (Eds): Springer-Verlag, pp. 415–438 (1997)
3. van Leeuwen, J.: Plability in Actions Videogames. Gamasutra Game Developer. <http://gamasutra.net/playability.html>. Accedido el 13 de Febrero de 2008